



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
CAMPUS PAU DOS FERROS
BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

**CLASSIFICAÇÃO DA QUALIDADE DE VINHOS TINTOS UTILIZANDO k-NN
COM BASE EM ATRIBUTOS FÍSICO-QUÍMICOS**

JOSÉ FERREIRA SOUSA NETO
TÚLIO GOMES DE ARAÚJO FEITOSA

Pau dos Ferros – RN

Maio de 2026

JOSÉ FERREIRA SOUSA NETO

TÚLIO GOMES DE ARAÚJO FEITOSA

**CLASSIFICAÇÃO DA QUALIDADE DE VINHOS TINTOS UTILIZANDO k-NN COM
BASE EM ATRIBUTOS FÍSICO-QUÍMICOS**

Relatório apresentado na disciplina de Sistemas Inteligentes como requisito para compor parte da nota da segunda unidade. Professor: Dr. Pedro Thiago Valério de Souza.

PAU DOS FERROS – RN

Maio de 2026

Sumário

1	Pré-processamento dos Dados	4
2	Treinamento e Seleção do Melhor k	4
3	Avaliação do Modelo	5
4	Discussão e Conclusão	6

1 Pré-processamento dos Dados

O conjunto de dados utilizado contém 1.599 amostras de vinho tinto da região do Vinho Verde (Portugal), com 11 atributos físico-químicos e uma variável-alvo denominada *quality*, originalmente expressa em escala inteira de 0 a 10. Conforme a atividade 1, foi realizada a recodificação da variável *quality* em três classes categóricas: *Ruim* (notas ≤ 5), *Médio* (notas 6 ou 7) e *Bom* (notas ≥ 8). Após a transformação, observou-se uma distribuição fortemente desbalanceada, com 744 amostras na classe Ruim, 837 amostras na classe Médio e apenas 18 amostras na classe Bom.

Em seguida, procedeu-se à normalização dos atributos utilizando o método Z-score, que transforma cada variável para média zero e desvio padrão unitário. A normalização é uma etapa obrigatória para o algoritmo k-NN porque este se baseia no cálculo de distâncias (geralmente distância euclidiana) entre os exemplos. Atributos com escalas muito distintas — como o teor alcoólico (aproximadamente 10–15) e a densidade (aproximadamente 0,99–1,00) — fariam com que variáveis de maior magnitude dominassem a distância, tornando as demais irrelevantes. A normalização garante que cada característica contribua igualmente para o cálculo da proximidade entre os pontos.

Posteriormente, os dados foram divididos em conjuntos de treino e teste na proporção de 80% e 20%, respectivamente, utilizando o parâmetro *stratify* para preservar a proporção das três classes em ambos os subconjuntos. O conjunto de treino ficou com 1.279 amostras, enquanto o conjunto de teste ficou com 320 amostras. As proporções entre as classes foram mantidas adequadamente, com cerca de 46,5% de amostras ruins, 52,3% de médias e 1,1% de boas em ambos os conjuntos.

2 Treinamento e Seleção do Melhor k

Foi empregado o algoritmo k-NN implementado na biblioteca *scikit-learn*, utilizando a distância euclidiana como métrica e submetendo o conjunto de treino ao processo de validação cruzada com 5 *folds*. Foram testados diferentes valores de k: 1, 3, 5, 7 e 11. A acurácia média obtida para cada valor é apresentada na Tabela 1.

Valor de k	Acurácia Média (5-fold CV)	Desvio Padrão
1	0,7295	$\pm 0,0110$
3	0,7044	$\pm 0,0141$
5	0,7084	$\pm 0,0093$
7	0,7029	$\pm 0,0208$
11	0,7068	$\pm 0,0157$

Tabela 1: Resultados da validação cruzada para diferentes valores de k

Observa-se que o melhor desempenho foi alcançado com $k = 1$, obtendo acurácia média de 0,7295. Para valores muito baixos de k (especialmente $k=1$), o modelo tende a se tornar complexo e com alta variância, sendo sensível a ruídos e a possíveis exemplos discrepantes, o que pode levar ao *overfitting*. No entanto, neste conjunto de dados específico, o melhor resultado ainda foi com $k=1$, sugerindo que as fronteiras de decisão entre as classes são relativamente bem definidas e que a inclusão de mais vizinhos introduz exemplos de outras classes que prejudicam a classificação.

Para valores muito altos de k (como $k=11$), o modelo torna-se mais simples e com maior viés, podendo suavizar excessivamente as fronteiras de decisão e perder detalhes importantes da distribuição dos dados. Observa-se que, à medida que k aumenta, a acurácia média apresenta ligeira oscilação, mas mantém-se em patamar inferior ao de $k=1$, confirmando que o aumento do número de vizinhos não trouxe benefícios para este problema.

3 Avaliação do Modelo

Com o melhor valor de k ($k=1$) selecionado, o modelo foi avaliado no conjunto de teste, reservado exclusivamente para esta etapa. Foram calculadas a matriz de confusão e as métricas de acurácia, precisão, recall e F1-score para cada uma das três classes. Os resultados obtidos são apresentados nas Tabelas 2 e 3.

Matriz de Confusão			
Valor Real \ Predito	Ruim	Médio	Bom
Ruim (0)	125	24	0
Médio (1)	29	138	0
Bom (2)	0	4	0

Tabela 2: Matriz de confusão do modelo k-NN ($k=1$) no conjunto de teste

Classe	Acurácia	Precisão	Recall	F1-score
Ruim (0)	0,8219	0,8117	0,8389	0,8251
Médio (1)	0,8219	0,8313	0,8263	0,8288
Bom (2)	0,9875	0,0000	0,0000	0,0000

Tabela 3: Métricas de desempenho por classe no conjunto de teste

A acurácia global do modelo no conjunto de teste foi de aproximadamente 82,19% (soma dos acertos na diagonal principal: $125 + 138 + 0 = 263$ acertos em 320 amostras). No entanto, a análise das métricas por classe revela um problema grave: a classe *Bom* (notas ≥ 8) não foi identificada corretamente em nenhuma amostra. O modelo classificou todas as 4 amostras da classe Bom como pertencentes à classe Médio, resultando em precisão, recall e F1-score iguais a zero para essa classe.

Essa falha é explicada pelo forte desbalanceamento do conjunto de dados: apenas 18 amostras da classe Bom no total (1,1% do dataset). Com um número tão reduzido de exemplos, o algoritmo k-NN não consegue formar "vizinhanças" representativas dessa classe, especialmente no espaço de características normalizado, onde os poucos exemplos da classe rara ficam isolados ou muito próximos a exemplos da classe majoritária.

4 Discussão e Conclusão

Com base nos resultados obtidos, pode-se responder diretamente às questões propostas:

1. **A normalização é obrigatória?** Sim, porque o k-NN baseia-se em distâncias. Atributos com escalas distintas distorcem o cálculo da proximidade, fazendo com que variáveis de maior magnitude dominem a decisão. A normalização Z-score aplicada garantiu que todos os atributos contribuíssem igualmente.
2. **O que acontece com valores muito baixos e muito altos de k?** Valores baixos ($k=1$) geram modelo complexo, com alta variância e sensibilidade a ruídos, mas que neste caso específico obteve o melhor desempenho. Valores altos ($k=11$) produzem modelo mais simples, com maior viés, suavizando fronteiras de decisão e reduzindo a acurácia.
3. **Entre quais classes o modelo erra com mais frequência?** Conforme a matriz de confusão, o modelo confunde predominantemente as classes *Ruim* e *Médio* (24 amostras ruins classificadas como médio; 29 amostras médias classificadas como ruins). A classe *Bom* é sistematicamente confundida com *Médio*, nunca sendo classificada corretamente.

Quanto à adequação do k-NN para este problema, a resposta é **parcialmente adequada**, com ressalvas importantes. O algoritmo apresentou desempenho razoável (acurácia $\approx 82\%$) para distinguir entre as classes majoritárias (Ruim e Médio). No entanto, o modelo falhou completamente na identificação da classe Bom, devido ao forte desbalanceamento e ao número extremamente reduzido de exemplos dessa classe.

Para aplicações práticas — como controle de qualidade na indústria vinícola ou certificação de denominação de origem — a incapacidade de identificar vinhos de alta qualidade seria um problema crítico (falsos negativos da classe Bom). Recomenda-se, em trabalhos futuros, a adoção de técnicas de balanceamento de classes (como SMOTE ou subamostragem), a coleta de mais amostras de vinhos de alta qualidade ou a experimentação com outros algoritmos mais robustos a classes desbalanceadas (como Random Forest ou XGBoost). Apesar das limitações, o k-NN demonstrou ser um modelo interpretável e de fácil implementação para a diferenciação entre vinhos de qualidade regular e baixa.